From One Simulation to a Toolkit

How the *Pallet Handling Simulation* (revC, PR99) is built,
and how to platformize it into a reusable digital-twin toolkit

ParaTech Platform R&D Team, ParagonCorp

Technical note, October 2026

Scope. This note reverse-engineers the single-file artefact `Pallet_Handling_Simulation_revC_PR99_5365.html` (28.8 MB) – its asset pipeline, runtime stack, discrete-event engine and authoring contracts – and proposes a toolkit architecture, scenario manifest and phased plan that turns this one-off build into a repeatable product for other packing lines, distribution-centre cells and vendor equipment.

Contents

1 Executive Summary **2**

2 How the Simulation Is Built **2**
2.1 Build pipeline: from CAD to a single HTML file 2
2.2 Runtime architecture 3
2.3 The 3D asset contract 3
2.4 Process model of the cell 5
2.5 Simulation engine 5
2.6 Limits of the current build 5

3 Platformizing into a Toolkit **6**
3.1 Design principles 6
3.2 Target architecture 7
3.3 Component responsibilities 7
3.4 The scenario manifest 7
3.5 Data flow with the manifest in the loop 8
3.6 From single sample to decision-grade statistics 8
3.7 Asset pipeline improvements 8

4 Roadmap **9**
4.1 Risks and mitigations 9
4.2 Requirements checklist for a new scenario 9

5 Conclusion **10**

1 Executive Summary

The Pallet Handling Simulation is a real-time 3D digital twin of a pallet-handling cell: AMRs, a pallet changer, two Helix 3 Evo stretch wrappers, a PR99 four-way strapper, forklifts and truck bays. It compares two layout options (compact vs. buffered) and reports throughput, lead time and utilisation so that a layout and forklift count can be chosen. It ships as *one self-contained HTML file*: two compressed glTF models, two textures and a bundled JavaScript application (three.js plus the simulation, roughly 27 KB of minified app code).

Key observations.

- The build is *craft-made but well structured*: a Blender-authored model whose node names and `extras` tags form an implicit API, driven by a fixed-timestep (0.05 s) discrete-event engine built from four primitives (tween, lane/slot, load, agent).
- About 97 % of the file is model payload (∼492 k triangles per option). Code and scenario knowledge (lane tables, waypoints, parameters) are *hard-coded* in the minified bundle.
- Reuse today means copying the file and editing minified code. That does not scale.

Recommendation. Extract the engine into a scenario-agnostic *core*, move every piece of scenario knowledge into a validated *scenario manifest* (JSON) that is bound to a named-node *asset contract*, add a headless *batch runner* for multi-replication KPI studies, and wrap all of it in an authoring CLI and a viewer shell. Section 3 specifies the target; Section 4 phases the work; the first milestone reproduces revC bit-for-bit from a manifest.

2 How the Simulation Is Built

2.1 Build pipeline: from CAD to a single HTML file

Figure 1 shows the end-to-end path. Equipment is modelled in Blender (vendor CAD imported or re-modelled), tagged with names and `extras.grp`, exported as glTF 2.0 (Khronos I/O 4.3, Y-up, metres), gzip-compressed, base64-encoded and inlined into `<script type="application/octet-stream">` blocks. The JavaScript bundle and two small PNG textures are inlined beside them.

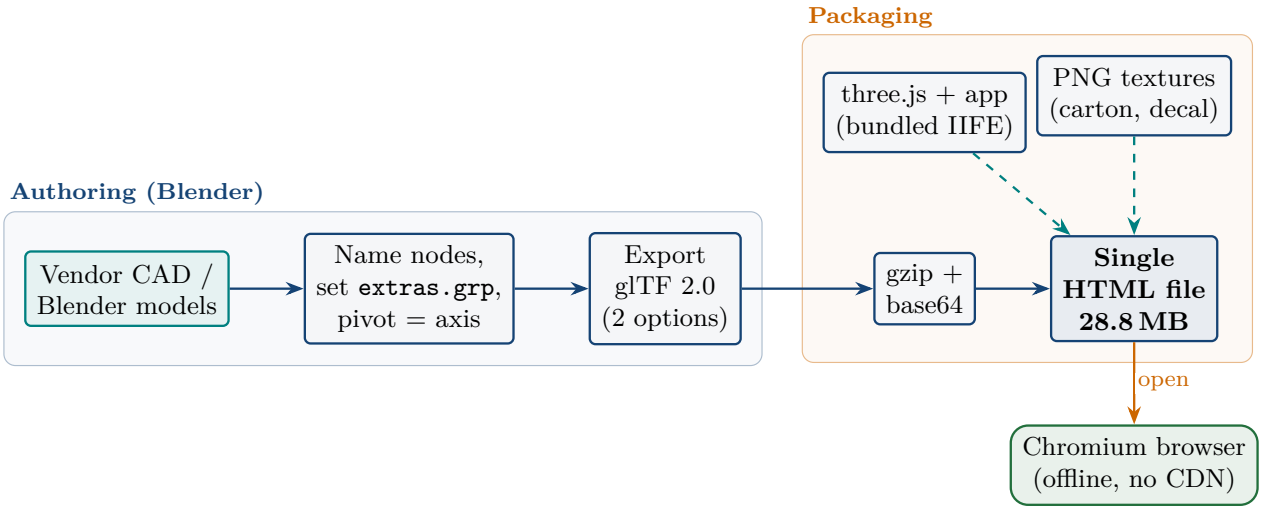


Figure 1: Build pipeline of revC. Solid arrows: model payload; dashed: code and textures inlined at packaging.

Region (lines)	Size	Content
1–68	small	HTML head and CSS (dark theme, fixed panes)
69–98	∼41 KB	DOM: canvas, control panel, KPI pane, loader, tag overlay, inline logos
100 / 101	∼14 MB each	`glb1`, `glb2`: Option 1 / Option 2 models (gzip + base64)
102	∼99 KB	`window.TEX`: carton and trailer-decal PNG data URIs
103–end	∼0.4 MB	Bundled IIFE: three.js (GLTFLoader, OrbitControls, PMREM) + simulation

Table 1: Anatomy of the 28.8 MB file.

2.2 Runtime architecture

At load time, the browser decodes `atob` → `DecompressionStream("gzip")` → `GLTFLoader.parse`. Only Option 1 is parsed on start; Option 2 is parsed lazily on first selection. Figure 2 separates the four runtime layers and the single coupling point between them: the *named-node lookup*.

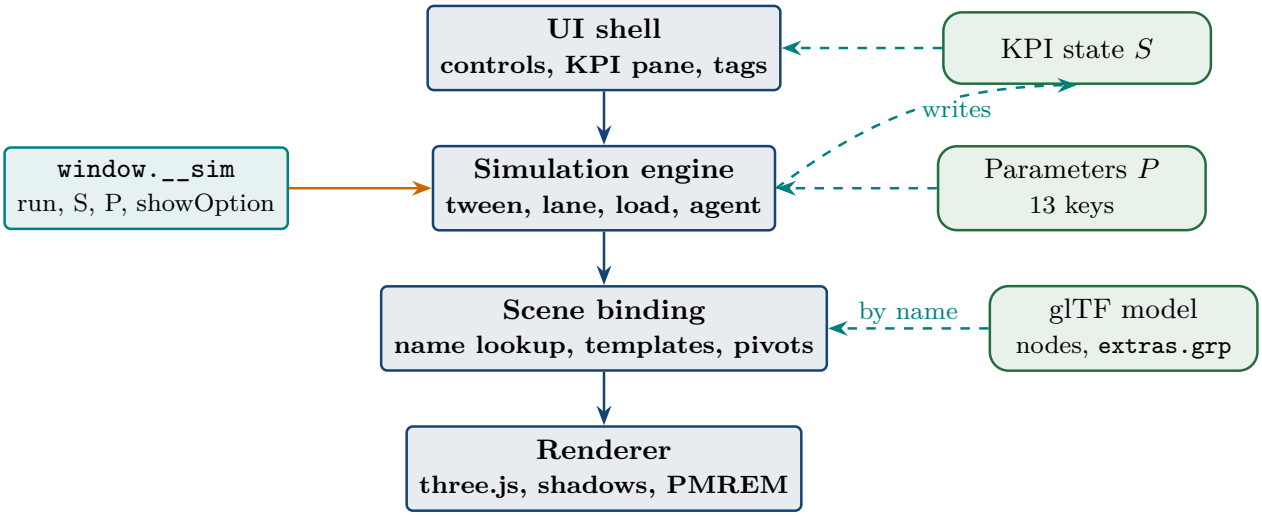


Figure 2: Runtime layers. The headless hook `window.__sim` is the seed of the batch runner.

Rendering. perspective camera (FOV 40°), ACES-style tone mapping (exposure 0.85), one warm directional light with a 4096^2 shadow map, a weak hemisphere fill, and a *procedural* image-based environment (a small room of emissive boxes through PMREM) – no HDRI file. Sim space is 2-D (x, y) on the floor, mapped to three.js as $(x, h, -y)$, in metres.

2.3 The 3D asset contract

Each option is a glTF of ∼24.7 MB uncompressed: ∼492 k triangles, 569–600 nodes, ∼350 meshes, ∼137 materials, *zero animations and zero skins*. All motion is code-driven. The code and the model are bound by five conventions (Figure 3):

1. **Name-based lookup** (e.g. `RotatingArm_PIVOT`, `FilmCarriage_Z`, `PR99_Sword`, `Forklift_Outside_Truck`). A rename breaks the sim silently.
2. **Layer tags** via `extras.grp`: `pc, line1, line2, fence, loads, walls, forklifts, buffer, trucks, floor`.
3. **Pivot = animation axis.** Origin at the pivot; the rest pose is stored on load and animation is relative to it.
4. **Hidden templates, cloned at runtime** (loads, AMR, forklift) and re-based to bottom-centre.
5. **Materials keyed by name** and re-skinned in code (wood, film, floor, carton logo, decal).

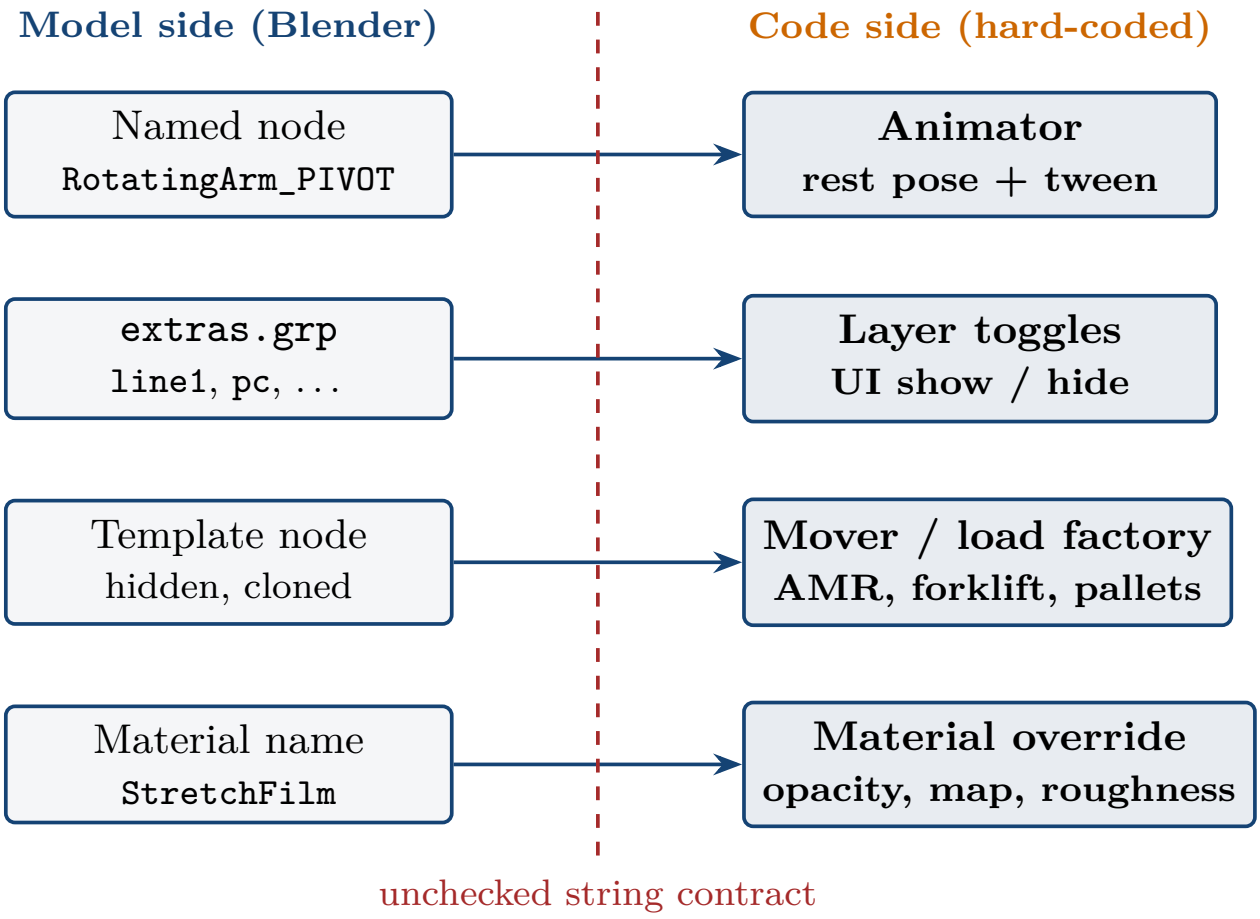


Figure 3: The implicit model–code contract. Every arrow is a string match that nothing validates.

2.4 Process model of the cell

The simulated cell moves one product from production to a truck. Figure 4 shows the flow including the routing decision (tall loads to Line 1 with strapping, short loads to Line 2).

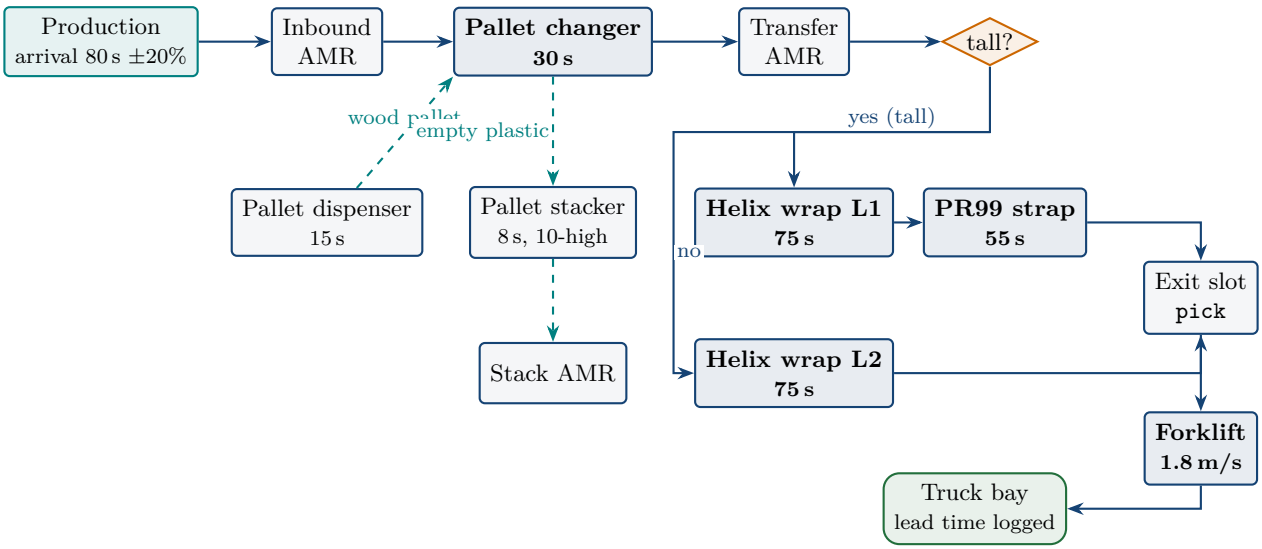


Figure 4: Material flow of revC. Solid: loaded product; dashed: empty pallets. Times are default concept assumptions.

2.5 Simulation engine

The engine is a *fixed-timestep discrete-event/time-stepped hybrid*: simulated time advances in $\Delta t = 0.05$ s steps; the render loop accumulates realDelta × speed (real delta clamped to 0.1 s, at most 2000 steps per frame) and steps the engine as many times as needed. Randomness is a single Lehmer LCG (seed 7, multiplier 16807 mod $2^{31} - 1$), reset on every Restart, which makes Option 1 vs. Option 2 runs directly comparable.

Four primitives carry the whole model:

Primitive	Semantics
Tween $(d, \text{update}, \text{done})$	Timed machine animation; every machine cycle is a tween advanced by Δt.
Lane $(\text{name}, y, \text{slots})$	Ordered slots $\{x, \text{proc}, \text{occ}, \text{res}\}$; `proc` (`wrap, strap, pick, amr, hold, disp, stack`) runs when a load arrives.
Load $\{\text{kind}, H, \text{tall}, \text{born}, \dots\}$	`product, wood, plastic, stack`; `born` gives lead time.
Agent (AMR / forklift)	Command queue of `path`, `lift`, `wait`, `call`; turn-in-place at 1.6 rad/s, then drive; carries one load.

Table 2: Engine primitives.

Lane rule. Scanning from exit backwards, a `ready` and not `moving` load advances when the next slot is free and unreserved, taking distance/v_{conv} seconds ($v_{conv} = 0.25$ m/s). Conflicts between movers are avoided *only* through slot reservation.

KPIs. Throughput per hour (after 15 min; rolling 60 min past 1 h), average lead time over the last 200 loads, station busy/blocked/idle fractions, queue at production, loads waiting for forklift, forklift utilisation = busy/simTime.

2.6 Limits of the current build

- Scenario knowledge (lane slot tables, waypoint polylines, home positions, stations) is hard-coded in minified code.

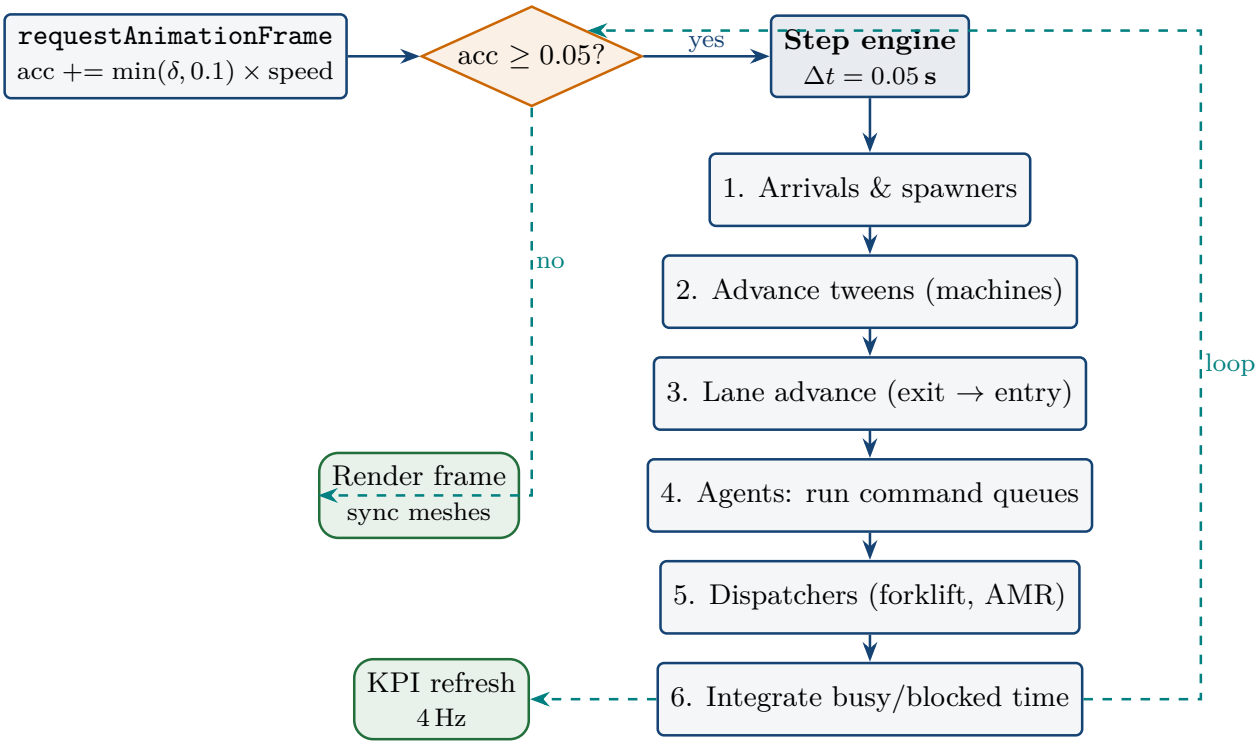


Figure 5: One frame of the engine loop. The loop is deterministic given seed, parameters and layout.

- Fixed hand-written waypoints, no path-finding or collision avoidance; machine state is boolean.
- One RNG stream; no warm-up period or replications, so KPIs are single-sample.
- 492 k triangles per option, 28 MB single file; names-as-API breaks silently.
- Parameters are concept assumptions, not calibrated to measured cycle times.

3 Platformizing into a Toolkit

3.1 Design principles

1. **Separate what changes from what does not.** Engine primitives are stable; layouts, lanes, cycle times and models change per scenario.
2. **Data over code.** Every hard-coded table in revC becomes a field in a validated manifest.
3. **One contract, checked at build time.** The names-as-API contract is declared once and verified against the glTF before any run.
4. **Simulation is not rendering.** The engine runs headless in Node/Worker; the 3D viewer is an optional consumer of its state.
5. **Reproducible by default.** Seeded streams, replications, versioned manifests and models.

3.2 Target architecture

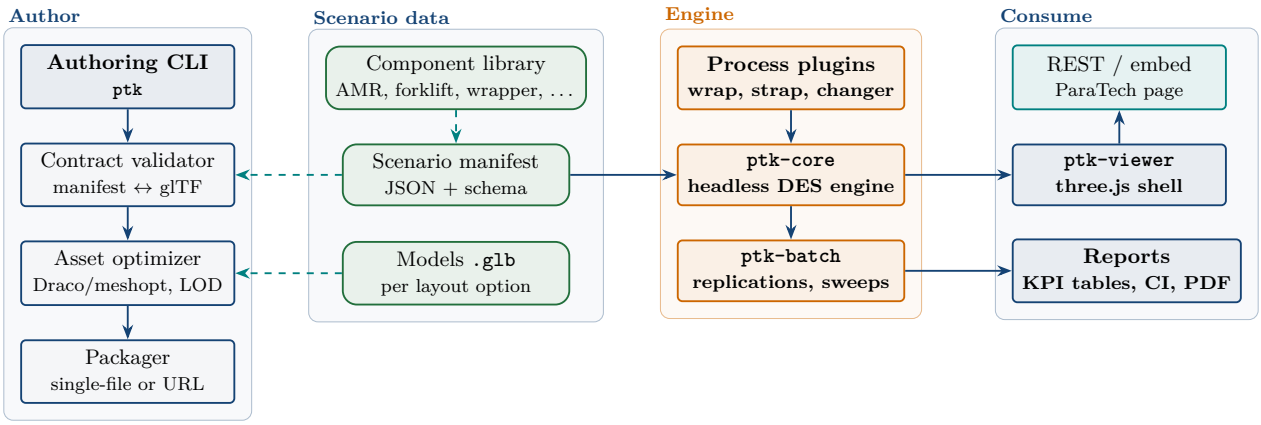


Figure 6: Toolkit architecture. Orange blocks are new engineering; the viewer and engine are extracted from revC.

3.3 Component responsibilities

Component	Responsibility	Extracted from revC
`ptk-core`	Time loop, RNG streams, tween, lane, load, agent, KPI accumulators; no DOM, no three.js	Engine layer (§2)
Process plugins	Station behaviours (`wrap`, `strap`, `changer`, `stack`); register by `proc` name	Process logic table
`ptk-viewer`	Scene binding, animators, camera presets, tags, KPI panes	Renderer + UI layers
`ptk-batch`	Seeded replications, warm-up, parameter sweeps, confidence intervals	`__sim.run`
Component library	Reusable machine/vehicle definitions: node names, pivots, travel, default cycle times	Asset catalogue
Validator	Fails the build if a manifest references a node, group or material the glTF lacks	(new)
Optimizer / packager	Draco/meshopt, decimation, split-asset or single-file export	Packaging step

Table 3: Toolkit components and their origin.

3.4 The scenario manifest

The manifest is the single source of scenario truth. It replaces the hard-coded lane tables, waypoints and parameter object. The excerpt below encodes Line 1 of Option 1 and one parameter from revC.

```
{
  "schema": "ptk.scenario/1",
  "model": { "glb": "options/compact.glb", "units": "m", "up": "Y" },
  "seed": 7,  "dt": 0.05,
  "lanes": [{
    "id": "L1", "y": -0.58,
    "slots": [
      { "x":  2.90 }, { "x":  1.00 },
      { "x": -1.595, "proc": "wrap",  "uses": "helix_L1" },
      { "x": -3.80 },
      { "x": -5.20,  "proc": "strap", "uses": "pr99" },
      { "x": -6.71,  "proc": "pick" }
```

```
    ]}],
  "stations": [{
    "id": "helix_L1", "plugin": "helix3evo", "cycle": {"param": "wrap"},
    "parts": { "arm": "RotatingArm_PIVOT", "carriage": "FilmCarriage_Z" }
  }],
  "agents": [{ "id": "fork1", "type": "forklift", "template": "Forklift_Outside_Truck",
               "speed": {"param": "fl"}, "home": [-10.6, -3.2] }],
  "params": { "wrap": { "label": "Wrapping cycle", "default": 75,
                        "min": 40, "max": 150, "step": 5, "unit": "s" } },
  "kpis": ["throughput_h", "lead_time_min", "util.station", "util.forklift"]
}
```

3.5 Data flow with the manifest in the loop

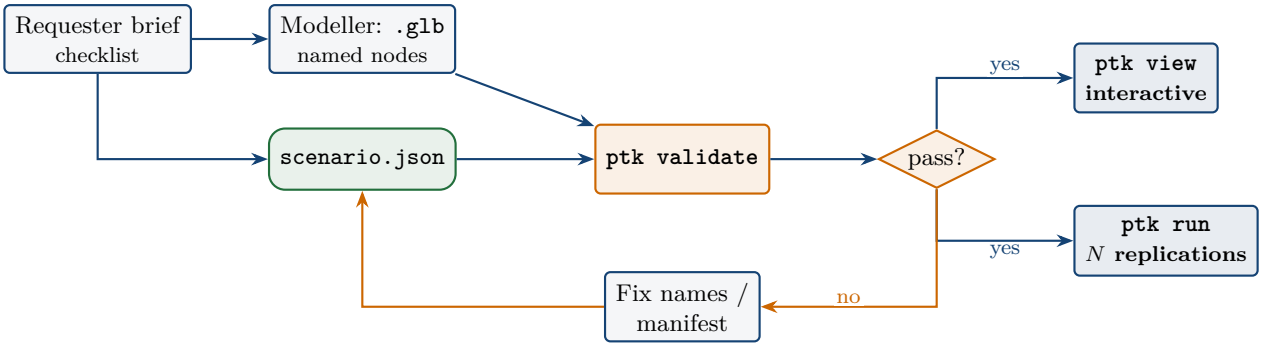


Figure 7: Authoring loop. The validator turns silent name mismatches into build-time errors.

3.6 From single sample to decision-grade statistics

revC reports a single stochastic trajectory. For a layout decision the toolkit should report, per option and parameter set, the mean and a confidence interval over R independent replications after a warm-up of length T_w. With replication means $\bar{X}_1, \ldots, \bar{X}_R$ (e.g. throughput per hour),

$$\hat{\mu} = \frac{1}{R}\sum_{r=1}^{R} \bar{X}_r, \qquad \hat{\mu} \pm t_{R-1,\,1-\alpha/2}\,\frac{s}{\sqrt{R}}.$$

Replications use independent seeded streams (split arrival, forklift-away and routing streams), and Option A vs. Option B are compared with *common random numbers* so that the paired difference isolates the layout effect. Because the engine is deterministic given `(manifest, seed)`, every reported number is reproducible from a hash of those two inputs.

3.7 Asset pipeline improvements

Concern	revC	Toolkit
Model delivery	Base64 inline, 28 MB	Separate `.glb` (Draco/meshopt), lazy per option; single-file export only for e-mail hand-off
Triangle budget	∼492 k per option	Target < 150 k, LOD for distant machines
Contract	Implicit string names	Component library declares names; validator enforces
Textures	Inline PNG data URIs	Separate files, KTX2 where useful
Hosting	Static file in an iframe, no file-level access control	Versioned assets behind the platform's auth

Table 4: Asset pipeline: current vs. target.

4 Roadmap

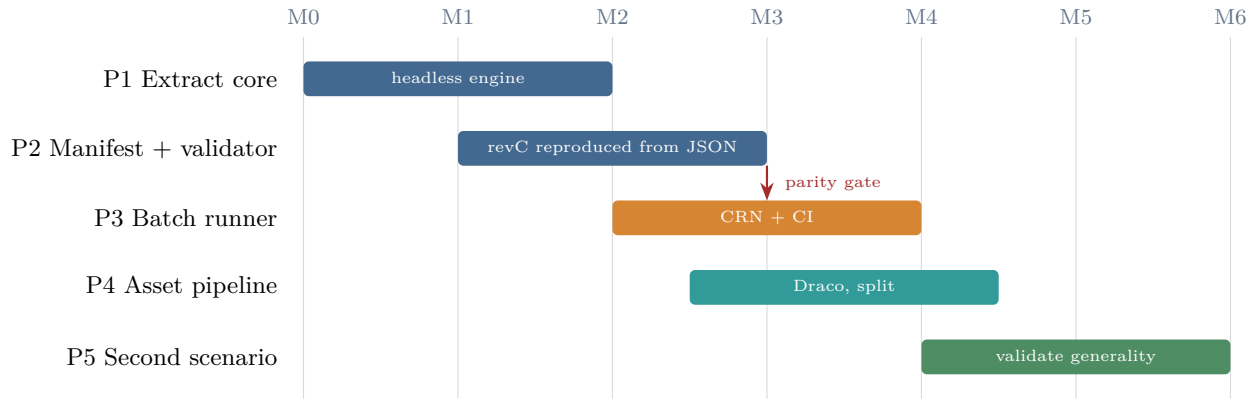


Figure 8: Indicative phasing (months). The parity gate: the manifest-driven engine must reproduce revC KPIs for the same seed before new features land.

Phase	Deliverable	Exit criterion
P1	Engine extracted to a headless module; viewer consumes its state	Same KPIs as `__sim.run` for seed 7
P2	Manifest schema, validator, revC encoded as `scenario.json`	Bit-identical KPI trace vs. P1
P3	Batch runner with warm-up, replications, CRN, sweeps	Option-1 vs. Option-2 report with confidence intervals
P4	Draco/meshopt pipeline, split assets, LOD	Load time and triangle budget met on a standard laptop
P5	Second scenario authored only from brief + manifest + model	No engine code changes required

Table 5: Phases and exit criteria.

4.1 Risks and mitigations

- **Behavioural drift during extraction** – keep the parity gate (seed-7 KPI trace) as a CI test.
- **Manifest becomes a programming language** – keep it declarative; put behaviour in plugins, not expressions.
- **Waypoints do not generalise** – start with authored polylines in the manifest; add grid path-finding later behind the same `path` command.
- **Uncalibrated parameters** – add a data-contract step to replace concept cycle times with measured distributions before external use.

4.2 Requirements checklist for a new scenario

1. **Layout:** floor footprint (m), walls and gates, axis orientation (Y-up, metres, sim $+y = -Z$), one compressed glTF per option.
2. **Equipment:** named node groups with layer tags; each moving part with origin on its pivot, a rest pose and a travel.
3. **Process:** stations (position, cycle, capacity, blocking rule), lane/slot map, routing rules, mover specs, arrival process, initial state, seed.

4. **Outputs:** exact KPI definitions, exposed parameters (min, max, step, default), camera presets, acceptance targets.

5 Conclusion

revC proves the pattern: a Blender-authored model, a small deterministic engine and a thin viewer are enough to compare layouts credibly. What it lacks is not capability but *separation*. By pulling scenario knowledge into a validated manifest, the engine into a headless core and statistics into a batch runner, the same investment yields a toolkit in which a new packing line is a brief, a model and a JSON file – not a fork of a 28 MB file.

Source note. Facts about revC were read from the file itself (embedded glTF payloads and the minified app script) and from the companion breakdown `pallet_handling_simulation_breakdown.md`. The toolkit architecture, manifest schema and roadmap are proposals, not existing software.